

TECHNICAL REFERENCE & CODEBASE AUDIT

OMS Label Lifecycle Reference

How labels are created, stored, controlled, and used across the OMS

Technical reference based on a static codebase audit on 21 August 2026.

Scope: QC serial labels; PIS, Product Database, Master and QC barcode values; shipping marks; EAN artwork; flat-carton and 3D-carton artwork; satin-label files; the Labels permission module; and all functional consumers.

Bottom line: the OMS uses the word label for three different data paths. QC serial labels are allocated numbers that are consumed once. Barcode labels are Item/QC business values. Shipping-label artwork is a file-object workflow. Do not treat them as one feature.

Audit method and scope

- 550 project files were scanned, excluding dependency folders, Git metadata, generated builds and coverage output.
- 238 files matched label, barcode, QR, shipping-mark or marking terms.
- Every functional match was traced through the relevant model, controller, route, service, frontend screen, script, test and current project documentation.
- Ordinary HTML form labels and UI captions were classified during the scan but excluded from the operational lifecycle because they do not create or persist label data.
- This is a source-code reference. It describes implemented behavior, not production records or external object-storage contents.

End-to-end flow chart

The diagram separates the three label lifecycles. Subsequent sections provide the complete field, rule, API and consumer inventory.

OMS label lifecycle — three independent data paths

Serial QC labels are not barcode values; barcode values are not uploaded shipping-label artwork.

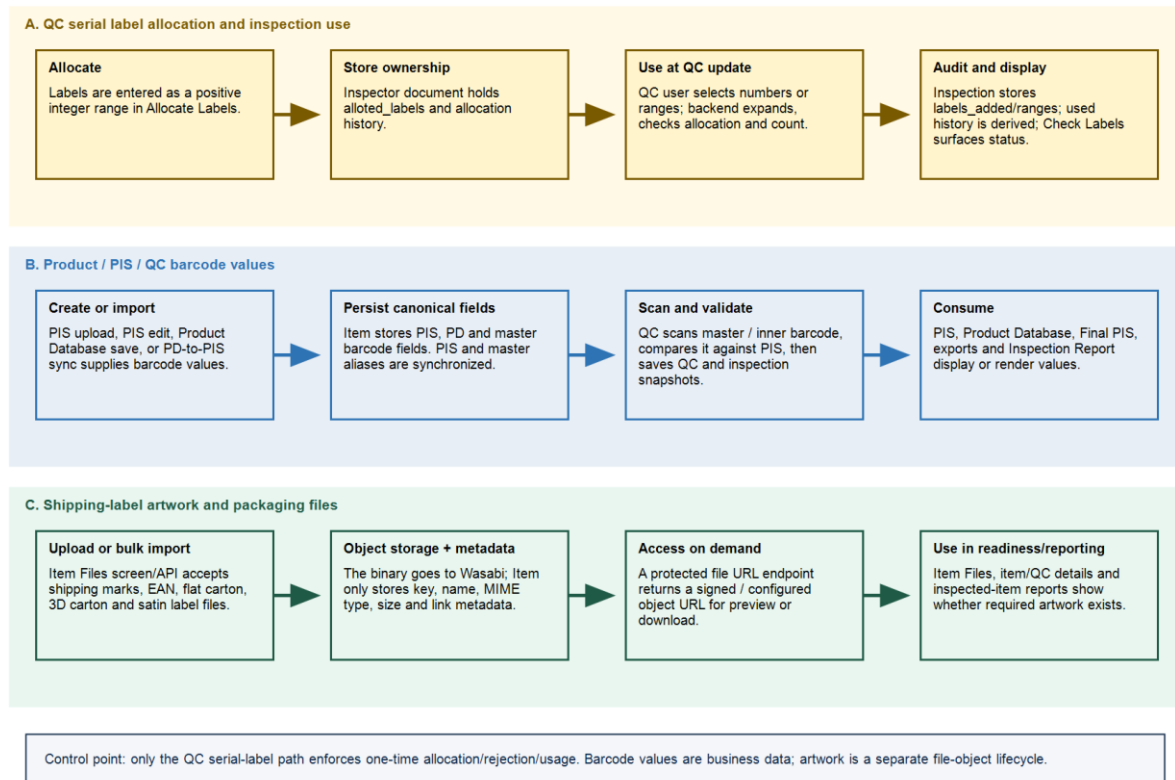


Figure 1. QC serial labels, barcode values, and artwork files have separate sources of truth and separate persistence paths.

1. Label taxonomy

1.1 QC serial labels

Examples: numeric labels such as 101 through 105; Inspector.alloted_labels; Inspection.labels_added.

Created by: an authorized user enters an inclusive positive-integer range in Allocate Labels, or uses the allocation API.

Used by: QC update, inspection history, Check Labels, allocation and usage statistics, inspection report label ranges, transfer/rejection flows.

1.2 PIS barcode labels

Fields: pis_barcode, pis_master_barcode and pis_inner_barcode on the Item document.

Created by: PIS workbook import, Edit PIS, Product Database-to-PIS synchronization, item import and operational scripts.

Used by: QC barcode defaulting/validation, PIS/PIS Diff/Final PIS comparisons, exports, Item/Master pages, update logs and visual barcode rendering in the Inspection Report.

1.3 Product Database and Master barcodes

Product Database fields: `pd_barcode`, `pd_master_barcode`, `pd_inner_barcode`, `pd_history` and `pd_checked`.

Master fields: `master_barcode`, `master_master_barcode` and `master_inner_barcode`.

Created by: Product Database save/check/approve workflow, PIS Diff/master update paths and selected import/sync utilities.

Used by: Product Database review, Item Masters, Final PIS and PIS sync. These fields are deliberately separate from PIS fields until a synchronization route copies data.

1.4 QC and Inspection barcode snapshots

QC fields: `barcode`, `master_barcode` and `inner_barcode`. Inspection uses the same field names as visit-level snapshots.

Created by: QC update or inspection-record edit.

Used by: Inspection Report, historical inspection detail, PIS versus inspected comparisons, barcode mismatch warnings and item QC summary synchronization.

1.5 Shipping-label artwork and packaging files

Fields: `shipping_marks.files`, `shipping_marks.ean`, `shipping_marks.flat_carton`, `shipping_marks.three_d_carton` and `satin_label`. Legacy fields `shipping_marks_1/2` and `flat_carton_1/2` are still read for compatibility.

Created by: Item Files upload API/UI or the bulk shipping-mark importer.

Used by: Item Files, Item Details, QC Details, Inspection Report document context and Inspected Items readiness flags.

1.6 Presentational labels

Examples: HTML label elements, button captions, modal titles and CSS class names.

These are interface/accessibility elements only. They are not an operational label data store and are not part of the flows below.

2. Storage map

2.1 QC serial label storage

- MongoDB collection `inspectors`, model `backend/models/inspector.model.js`:

- `alloted_labels`: current assigned serial numbers. The spelling `alloted` is part of the existing schema/API.
- `used_labels`: denormalized current list of used serial numbers.
- `rejected_labels`: serials permanently rejected for that inspector.
- `labels_allotted_by`: user that last allocated/managed labels.
- `label_allocation_history`: audit records for `allocate`, `transfer_in`, `transfer_out`, `reject`, `replace` and `remove`. Each event includes `previous/next` values, linked inspectors, actor, timestamp and remarks.
- `label_used_history`: denormalized history rebuilt from Inspection records. It includes labels, inspection/QC/request IDs, order/brand/vendor/item metadata and use/update timestamps.
- MongoDB collection `inspections`, model `backend/models/inspection.model.js`:
- `label_ranges`: the range selections entered for a visit, as start/end pairs.
- `labels_added`: the final expanded numeric labels used during that visit.
- The Inspection record is the forensic source for actual used QC serial labels.
- MongoDB collection `qc`, model `backend/models/qc.model.js`:
- `labels`: an aggregate unique set across inspection records for the QC.
- This is refreshed after QC updates and inspection-record edits. It is not the source used to calculate `Inspector.used_labels`.

2.2 Barcode storage

- Item PIS state, `backend/models/item.model.js`:
- `pis_barcode`, `pis_master_barcode` and `pis_inner_barcode` are strings.
- The pre-validate hook synchronizes `pis_barcode` and `pis_master_barcode` as compatibility aliases.
- `barcode_exempted` is a boolean that disables otherwise required PIS barcode enforcement.
- Item Product Database state:
- `pd_barcode`, `pd_master_barcode` and `pd_inner_barcode` are strings.
- `pd_history`, `pd_checked` and the PD actor fields record workflow and audit state.
- Product Database helpers normalize `pd_barcode` and `pd_master_barcode` together.
- Item Master state:
- `master_barcode`, `master_master_barcode` and `master_inner_barcode` are strings.
- The pre-validate hook keeps `master_barcode` and `master_master_barcode` synchronized.
- Item QC summary:
- `qc_barcode`, `qc_master_barcode` and `qc_inner_barcode` are strings stored under the Item document.
- The primary/master barcode fields synchronize.
- QC record:
- `barcode`, `master_barcode` and `inner_barcode` are numeric schema fields.
- The QC pre-validate hook uses `master_barcode` as the primary source and mirrors it into `barcode`.
- Inspection record:
- `barcode`, `master_barcode` and `inner_barcode` are strings.
- They are copied as a visit-level snapshot when an Inspection row is created or edited.

2.3 Shipping artwork storage

- MongoDB Item document stores file metadata only: key, originalName, contentType, size, link and public_id.
- The binary file is stored in Wasabi-compatible object storage, normally under the item-shipping-marks folder.
- createStorageKey() builds sanitized, nondeterministic object keys using the folder, timestamp, random 8-byte hexadecimal suffix and file base name.
- getObjectUrl() returns a configured public URL when available; getSignedObjectUrl() produces protected time-limited read URLs otherwise.
- The protected Item file endpoint returns the usable URL. The frontend should not construct storage URLs itself.

3. QC serial label lifecycle

3.1 Create and allocate

1. Navbar exposes Allocate Labels when the user has labels.manage or labels.assign in the frontend permission context.
2. client/OMS/src/components/AllocateLabelsModal.jsx accepts a start and end number and expands the inclusive range locally.
3. It calls PATCH /inspectors/:id/allocate-labels with a labels array.
4. backend/controllers/inspector.controller.js parses only positive integers, deduplicates and sorts them.
5. The controller refreshes used state from Inspection records, then blocks:
 - labels already used by the selected inspector;
 - labels rejected by the selected inspector;
 - labels allocated to another inspector;
 - labels used by another inspector; and
 - labels rejected by another inspector.
6. New values are appended to Inspector.alloted_labels; labels_allotted_by is set; action=allocate is appended to label_allocation_history.

The code does not generate a serial sequence automatically. The authorized user supplies the numbers. The application validates, stores and audits them.

3.2 Transfer, reject, replace and remove

- PATCH /inspectors/transfer-labels moves unused serials between inspectors. It writes paired transfer_out and transfer_in history entries. Used, rejected and conflicting values are refused.
- PATCH /inspectors/:id/reject-labels moves currently allocated, unused serials from alloted_labels to rejected_labels. Rejected values cannot be allocated or used again.
- PATCH /inspectors/:id/replace-labels validates conflicts before replacing an inspector allocation and writes action=replace history.
- DELETE /inspectors/:id/labels detaches selected allocated values and writes action=remove history.

- GET /inspectors/:id/label-usage supports summary or full detail. Full detail includes allocated, used, rejected and unused lists plus both histories.

3.3 Use during QC update

1. UpdateQcModal submits direct labels, label_ranges, QC quantities and barcode validation state to PATCH /qc/update-qc/:id.
2. qc.controller accepts direct label arrays or expands every start/end range. A range must have both integer endpoints, non-negative values and start less than or equal to end.
3. New labels are checked against the selected inspector allocation. The backend blocks rejected, unallocated and already-used labels.
4. Unless the actor is label-exempt, labels and passed quantity require qc_checked greater than zero for the visit.
5. Unless label-exempt, the label count must match the required count:
 - Carton mode: passed quantity x 1.
 - Other packing modes: passed quantity x number of box-size entries.
 - At least one box size is required whenever the count rule applies.
6. qc.labels receives the unique aggregate set.
7. upsertInspectionRecordForRequest stores normalized label_ranges and the expanded labels_added array on the current Inspection record.
8. When a record is rewritten/edited, the aggregate QC labels are rebuilt from the saved Inspection rows.

3.4 Deriving use state

syncInspectorUsedLabelsFromInspectionRecords queries Inspection documents for the inspector user, flattens labels_added, deduplicates/sorts the values and writes Inspector.used_labels. It then rebuilds label_used_history from the same records.

Consequently:

- Inspection.labels_added is the reliable audit/reconciliation source for use.
- Inspector.used_labels is current denormalized convenience state.
- Inspector.label_used_history is denormalized UI/report data, not an independent label-allocation source.

3.5 QC serial-label exceptions

- backend/helpers/labelExemptUsers.js reads comma-separated user IDs from LabelExemptUsers or LABEL_EXEMPT_USERS.
- The frontend mirrors this environment configuration in client/OMS/src/utis/qcUpdateAccess.js.
- Label-exempt users bypass the QC serial-label count rule.
- Label exemption is not barcode exemption. Item.barcode_exempted controls PIS barcode requirements.

4. Barcode label lifecycle

4.1 Creation sources

- PIS workbook upload:
- POST /items/:itemId/pis-upload uses parseAndSyncPisUpload.middleware.
- Parsed master/PCS values populate pis_master_barcode plus pis_barcode alias, and pis_inner_barcode where present.
- The uploaded workbook is separately converted/stored as PIS file metadata; that is distinct from the barcode value itself.
- Manual PIS edit:
- client/OMS/src/components/EditPisModal.jsx calls PATCH /items/:id/pis.
- The standard path writes PIS fields. PIS Diff/master-update mode can write Master fields and set pis_checked_flag instead.
- Product Database workflow:
- client/OMS/src/pages/ProductDatabase.jsx calls PATCH /items/:id/product-database, POST check or POST approve.
- backend/helpers/productDatabase.js normalizes primary/master values and records PD state/history.
- Product Database-to-PIS synchronization:
- POST /items/:id/pis/sync-product-database and POST /items/pis/sync-product-database copy available PD master/inner barcode values into the PIS fields and record synchronization metadata/logs.
- Operational imports and scripts:
- Item import code, PIS extractor/sync utilities and updatePISbarcode.js can create or migrate barcode values outside the interactive UI.

4.2 QC scan and validation

The reference requirement is PIS data unless Item.barcode_exempted is true.

- Individual validation:
- the master/individual scanned value is compared with pis_barcode.
- Inner + master validation:
- master is compared with pis_master_barcode;
- inner is compared with pis_inner_barcode;
- it is only valid for an appropriate carton/inner-master packing mode.
- For QC users, a changed master or inner barcode must be scanned rather than manually entered. The validation type must be selected and each required scanned value must match PIS.
- resolveBarcodeWithPisDefault:
- uses the stored PIS barcode when the submitted/current QC value is blank or zero;
- retains a populated non-zero QC value when no new blank override is sent.
- POST /qc/scan-barcode:
- requires qc.edit at the route layer;
- controller additionally restricts upload decoding to admins or label-exempt users;

- accepts JPG, JPEG, PNG, WEBP and PDF;
- PDF input is rasterized page-by-page, capped at five pages;
- the service tries auto rotation, upscale and inversion variants;
- ZXing is configured for CODE_128, EAN_13, EAN_8, UPC_A, UPC_E, ITF and CODABAR;
- response returns numeric barcode, raw text, detected format, source type and page number.

4.3 Display and visual rendering

- backend/helpers/barcodeFormat.js and client/OMS/src/utls/barcode.js normalize numeric values for display.
- A usable 12-digit value is left-padded to 12 digits and receives an EAN-13 check digit.
- Blank or all-zero values display as Not Set.
- Values that cannot be represented as EAN-13 remain text.

client/OMS/src/pages/inspection_report.jsx imports react-barcode:

- valid EAN-like values are rendered as EAN13;
- other values are rendered as CODE128;
- the report presents PIS and QC values, visually renders the barcode, flags mismatches and displays QC serial-label ranges;
- report HTML can be exported through the central PDF system.

client/OMS/package.json declares both react-barcode and jsbarcode. The audited source uses react-barcode directly in the Inspection Report; no direct jsbarcode import was found.

5. Shipping-label artwork and EAN files

5.1 File types

- shipping_marks: multiple PDFs/JPG/JPEG/PNG files at shipping_marks.files.
- ean: one PDF/JPG/JPEG/PNG file at shipping_marks.ean.
- flat_carton: multiple PDF/JPG/JPEG/PNG files at shipping_marks.flat_carton.
- three_d_carton: one PDF/JPG/JPEG/PNG file at shipping_marks.three_d_carton.
- satin_label: one PDF at Item.satin_label.

Satin Label is available only when satin_label_required is true. The normal item create/edit forms set this flag. The bulk shipping-mark importer sets it before uploading a satin-label file when necessary.

5.2 Upload and persistence

1. Item Files uses client/OMS/src/constants/itemFiles.js to select type, allowed extensions/MIME types, preview mode and eligibility.
2. POST /items/:id/files requires images_documents.upload. The route accepts one file or files arrays.
3. backend/controllers/item.controller.js checks item access, file type, cardinality, MIME/extension and conditional eligibility.
4. uploadBuffer writes each binary to Wasabi-compatible storage.

5. The Item field receives metadata, prior file keys are scheduled for deletion when replaced, update history is appended and security activity is recorded.

5.3 Retrieval and deletion

- GET /items/:id/files/:fileType/url requires images_documents.view and returns a signed/configured URL.
- Item Files previews image/PDF/office files and chooses in-app versus external opening based on device/preview mode.
- DELETE /items/:id/files/:fileType requires images_documents.delete.
- Deletion first removes database metadata and then attempts object deletion. A storage cleanup failure is returned as a warning, so operational follow-up is required for potential orphaned objects.

5.4 Bulk importer

backend/scripts/uploadShippingMarksFolderViaApi.js recognizes shipping-mark folders and filenames, derives item code/type, groups multiple files per item/type and uses the protected Item APIs.

Its focused test maps:

- shipping marks;
- EAN;
- flat carton;
- 3D carton; and
- satin label.

5.5 Consumers

- Item Files page: list, upload, retrieve, preview and delete.
- Item Details and QC Details: shared metadata/URL helpers for contextual file display.
- Inspection Report: attached shipping-mark, satin, EAN and carton context near inspection data.
- Inspected Items Report: file presence becomes readiness state such as Shipping Marks Uploaded and satin-label readiness. The report does not read the binary file contents.
- wasabiStorage.service: creates object keys; performs upload, protected URL generation, metadata checks and object deletion. MongoDB never stores the artwork binary.

6. Access control and API map

6.1 Labels permission module

backend/helpers/permissions.js defines the Labels module with the labels key. Backend enforcement is requirePermission(module, action), which resolves effective role permissions through permission.service.

- GET /inspectors/options requires labels.view.
- All other Inspector routes require labels.manage.
- Navbar shows Check Labels and Allocate Labels when labels.manage OR labels.assign is true in frontend context.

- Important mismatch: labels.assign alone can make a UI action visible, but current Inspector routes still require labels.manage. Do not grant assign-only access expecting it to work until the UI condition or route policy is aligned.
- Default builders grant broad permissions to admin-like roles and dev; user/QC default permission sets do not include Labels actions.
- Runtime RolePermission records can override defaults. Backend route guards, not frontend visibility, are the security boundary.

6.2 Relevant API endpoints

- GET /inspectors/options: label-viewable inspector options.
- GET /inspectors: Check Labels allocation overview.
- GET /inspectors/:id: full inspector allocation detail.
- GET /inspectors/:id/label-usage: summary or full usage/history.
- PATCH /inspectors/:id/allocate-labels: append allocated QC serial labels.
- PATCH /inspectors/transfer-labels: transfer unused serial labels.
- PATCH /inspectors/:id/reject-labels: permanently reject allocated unused serials.
- PATCH /inspectors/:id/replace-labels: replace allocation after validation.
- DELETE /inspectors/:id/labels: remove selected allocations.
- PATCH /qc/update-qc/:id: consume QC serial labels and persist barcodes/inspection state.
- PATCH /qc/:id/inspection-records: edit historical label ranges, used labels and barcode snapshots.
- POST /qc/scan-barcode: decode uploaded barcode image/PDF for admin or label-exempt users.
- PATCH /items/:id/pis: update PIS barcode business data.
- PATCH/POST /items/:id/product-database and check/approve routes: Product Database barcodes.
- POST /items/:id/pis/sync-product-database and bulk equivalent: copy PD barcode values into PIS.
- POST /items/:id/files: upload artwork.
- GET /items/:id/files/:fileType/url: read artwork through protected URL generation.
- DELETE /items/:id/files/:fileType: remove artwork metadata and object.
- POST /items/:itemId/pis-upload: import PIS workbook, synchronize PIS data and store converted PIS PDF metadata.

7. Functional file index

7.1 Backend data, rules and storage

- backend/models/inspector.model.js – allocation/rejected/used arrays and allocation/use-history schemas.
- backend/models/inspection.model.js – per-visit label ranges, used serials and barcode snapshots.
- backend/models/qc.model.js – numeric QC barcode/master/inner values and primary/master alias synchronization.
- backend/models/item.model.js – PIS/PD/master/QC barcodes; shipping_marks and satin_label metadata; aliases.
- backend/controllers/inspector.controller.js – allocation, transfer, rejection, replace/remove, conflicts and derived use state.

- backend/controllers/qc.controller.js – serial range expansion/count rules, allocation checks, barcode validation/defaults and inspection persistence.
- backend/controllers/item.controller.js – PIS/PD barcode mutation/sync, comparison/export, file config, Wasabi persistence and CRUD.
- backend/helpers/barcodeFormat.js – EAN-13 normalization and display.
- backend/helpers/productDatabase.js – PD barcode normalization/defaulting/required-field rules.
- backend/helpers/labelExemptUsers.js – configured label-exempt IDs.
- backend/helpers/itemUpdateAudit.js – PIS/PD barcode audit labels and values.
- backend/middlewares/parseAndSyncPisUpload.middleware.js – PIS workbook barcode mapping.
- backend/services/qcBarcodeScan.service.js – barcode decoding pipeline.
- backend/services/wasabiStorage.service.js – object key, upload, signed URL and deletion service.
- backend/services/inspectionItemSync.service.js and backend/services/itemSync.js – synchronization consumers of Item/QC barcode state.
- backend/routers/inspector.routes.js – Labels permission boundary and inspector label routes.
- backend/routers/qc.routes.js – QC update and barcode-upload routes.
- backend/routers/items.routes.js – PIS, Product Database and item-file routes.
- backend/helpers/permissions.js, backend/services/permission.service.js and backend/middlewares/permission.middleware.js – Labels module, effective permissions and enforcement.

7.2 Frontend screens and utilities

- client/OMS/src/components/AllocateLabelsModal.jsx – range allocation, transfer/rejection and usage summaries.
- client/OMS/src/components/CheckLabelsModal.jsx – allocation search, ranges and history display.
- client/OMS/src/components/Navbar.jsx – label action visibility.
- client/OMS/src/components/UpdateQcModal.jsx – QC serial ranges, browser scan, upload decode and barcode validation.
- client/OMS/src/components/EditPisModal.jsx – PIS barcode payload and comparison display.
- client/OMS/src/components/EditInspectionRecordsModal.jsx – historical inspection-record label/barcode edit.
- client/OMS/src/components/TransferInspectionModal.jsx – transfer of inspection label evidence.
- client/OMS/src/pages/QcDetails.jsx – per-record label ranges and barcode context.
- client/OMS/src/pages/inspection_report.jsx – react-barcode rendering, mismatch warning, serial range display and file context.
- client/OMS/src/pages/PIS.jsx, PISDiffs.jsx and FinalPISCheck.jsx – PIS/master barcode review, comparison, export and update paths.
- client/OMS/src/pages/ProductDatabase.jsx, ProductDatabaseDetails.jsx and ItemMasters.jsx – PD/master barcode workflow and display.
- client/OMS/src/pages/ItemFilesPage.jsx, ItemDetails.jsx and InspectedItemsReport.jsx – artwork lifecycle, preview and readiness display.
- client/OMS/src/constants/itemFiles.js – file-type catalog, acceptance rules and item eligibility.
- client/OMS/src/utills/barcode.js – frontend EAN-13 display behavior.
- client/OMS/src/utills/qcUpdateAccess.js – label-exempt configuration and QC editing window.

- client/OMS/src/services/qcBarcode.service.js â€” upload decoder API call.
- client/OMS/package.json â€” react-barcode and jsbarcode package declarations.

7.3 Scripts, tests and documentation

- backend/scripts/uploadShippingMarksFolderViaApi.js â€” bulk artwork mapping/upload.
- backend/scripts/updatePISbarcode.js, syncPisWorkbooks.js and PIS_extractor.js â€” PIS/barcode maintenance and import utilities.
- backend/scripts/migrateItemQcBarcodeFieldsToString.js and migrateInspectionBarcodeFieldsToString.js â€” string-storage migrations.
- backend/tests/inspectorLabelAllocation.test.js â€” allocation conflict ownership and populated-user handling.
- backend/tests/itemBarcodeAlias.test.js â€” Item PIS alias synchronization and partial-save safety.
- backend/tests/pisBarcodeExemption.test.js and qcPisBarcode.test.js â€” exemption/defaulting behavior.
- backend/tests/uploadShippingMarksFolderViaApi.test.js and inspectedItemsReport.test.js â€” artwork mapping and satin-label readiness.
- docs/api-map.md â€” inspector, QC, PIS and Item-file endpoint/permission inventory.
- docs/PIS_PD_MASTER_ITEM_FLOW.md â€” existing PIS/PD/master data-flow reference.
- docs/PDF_EXPORT_SYSTEM.md â€” central PDF export and physical label-PDF guidance.
- docs/OMS_SOURCE_TREE.md, OMS_REPORTS_AND_FUNCTIONS_SUMMARY.md and MEASUREMENT_MISMATCH_COMPARISON_FLOW.md â€” supporting system/report/comparison references.

8. Observations and maintenance checklist

- Use exact terminology in future requirements: QC serial label, barcode value or shipping-label artwork.
- Audit used QC serials from Inspection.labels_added, not only Inspector.used_labels.
- Preserve the PIS/master alias rules when changing schema or migration code.
- Monitor both MongoDB metadata and Wasabi objects, especially storage-delete warnings.
- Resolve the labels.assign versus labels.manage UI/backend mismatch before assigning roles with only assign.
- The code renders barcodes inside the Inspection Report. No separate standalone print-label creation API was found in the audited source.
- Add end-to-end test coverage before changing allocation concurrency, range-count rules or object-store cleanup behavior.
- Update this reference whenever a new label type, barcode source, file type, permission or consumer is added.

Final answer: where are labels stored?

QC serial labels are stored in the Inspector and Inspection MongoDB collections, with QC holding an aggregate view. Barcode labels are stored on Item, QC and Inspection documents. Shipping-label artwork

binaries are stored in Wasabi-compatible object storage, while their metadata is stored on the Item document. Permission-checked backend APIs create, validate, transfer, retrieve and delete each category.